



Complemento Práctico: El Fin de los Arreglos Estáticos

Hasta ahora, si queríamos guardar 10 empleados, teníamos que crear un `Empleado[10]`. Pero, ¿qué pasaba si contratábamos al empleado número 11? El programa fallaba con un temido `ArrayIndexOutOfBoundsException`. Este problema es el de todos los días cuando trabajamos con estructuras de tamaño fijo que intentan modelar una realidad inherentemente dinámica.

En esta unidad introducimos **ArrayList**, una herramienta revolucionaria que soluciona este problema de raíz. Además, utilizaremos esta nueva capacidad para modelar la relación más común en la industria del software: **Uno a Muchos (1:N)**. Analizaremos cómo un objeto puede "ser propietario" de una lista de otros objetos y cómo gestionar esa relación sin perder datos ni comprometer la integridad de nuestro sistema.

Este material complementa el video de la Unidad 6 - Actividad 1, profundizando en los conceptos fundamentales que transformarán su forma de programar para siempre.

Contenido

Evolución del Arreglo

Por qué el arreglo estático quedó obsoleto para datos variables y cómo hemos evolucionado hacia soluciones más flexibles.

Anatomía de ArrayList

El "envoltorio inteligente" y el poder del Operador Diamante en la seguridad de tipos.

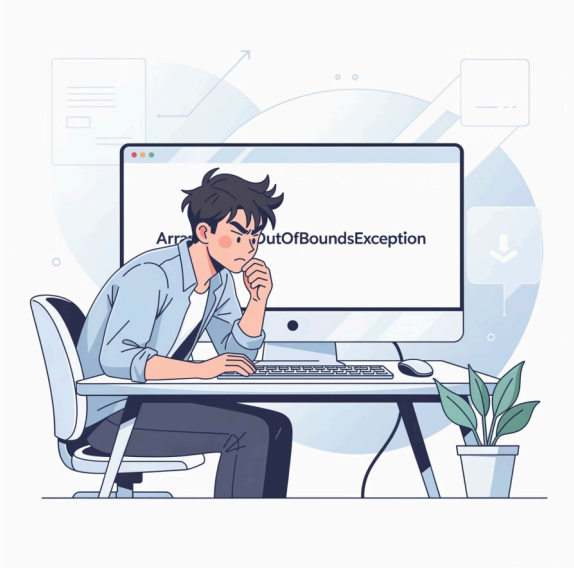
Arquitectura 1:N

Cómo diseñar correctamente una clase que contiene y gestiona una lista de objetos relacionados.

Métodos de Gestión

El patrón ABM: agregar, eliminar y listar elementos manteniendo la encapsulación.

La Limitación del Tamaño Fijo



El Problema Clásico

En el video se plantea el dilema fundamental: **"La vida real es dinámica"**. No sabemos con certeza cuántos alumnos se inscribirán en una materia, cuántos productos tendrá un carrito de compras, o cuántos empleados contratará un departamento en el próximo año.

Los arreglos tradicionales nos obligan a tomar decisiones imposibles:

- Si declaramos un tamaño pequeño, nos quedamos sin espacio
- Si declaramos un tamaño enorme, desperdiciamos memoria
- Cualquier cambio requiere recompilar el código

Esta rigidez hace que nuestros programas sean frágiles y poco adaptables a las necesidades reales del negocio.

ArrayList: El Arreglo Inteligente

La Solución: `java.util.ArrayList`

Piensen en el ArrayList como un **Arreglo Elástico**. Es una estructura que combina lo mejor de dos mundos: la eficiencia del arreglo tradicional con la flexibilidad de crecer dinámicamente según nuestras necesidades.

01

Internamente usa un arreglo

Tiene una capa de inteligencia (un "wrapper") que gestiona el arreglo subyacente

02

Crece automáticamente

Cuando se llena, crea un arreglo más grande en segundo plano, copia todo y descarta el anterior

03

Transparencia total

Para nosotros es invisible: simplemente llamamos a `.add()` y la lista crece sin límites aparentes

Esta abstracción nos libera de la gestión manual de memoria y nos permite concentrarnos en la lógica de negocio. El ArrayList maneja toda la complejidad interna, ofreciéndonos una interfaz simple y potente.

Seguridad de Tipos: El Operador Diamante

El Problema del Pasado

Una lista puede guardar objetos, pero *¿qué tipo de objetos?* En versiones antiguas de Java, podíamos mezclar Manzanas con Peras en la misma lista, lo que causaba errores catastróficos durante la ejecución. Imaginen intentar calcular el salario de lo que creían que era un Empleado, ¡pero resultaba ser un String!

La Sintaxis Moderna: Generics

Los **Generics** introdujeron seguridad en tiempo de compilación. Debemos ser explícitos sobre qué tipo de objetos almacenaremos:

```
// "Diamante" <Empleado>: Solo permite
guardar Empleados
// Si intento guardar un String, NetBeans
me da error
// antes de compilar
ArrayList<Empleado> lista = new
ArrayList<>();
```

- ❏ **Consejo profesional:** Nunca usen ArrayList "crudos" (sin el diamante). Siempre especifiquen el tipo: <String>, <Integer>, <Producto>. El compilador es su aliado.



Arquitectura 1:N: Uno a Muchos

Modelando la Relación "Tiene-Muchos"

El corazón de esta actividad es aprender a modelar correctamente un **Departamento** que tiene muchos **Empleados**. Esta es la relación más común en el desarrollo de software profesional, y dominarla es fundamental para su carrera.

Ya no usamos un atributo simple, sino una **Colección**. Observen el diseño correcto:

```
public class Departamento {  
    private String nombre;  
  
    // Relación 1 a N: Un depto TIENE UNA LISTA de empleados  
    private ArrayList<Empleado> empleados;  
  
    public Departamento(String nombre) {  
        this.nombre = nombre;  
        // ¡IMPORTANTE! Inicializar la lista en el constructor  
        // Si olvidamos esto, la lista es null y dará error  
        this.empleados = new ArrayList<>();  
    }  
}
```

¡Atención al Constructor!

El error más común de los principiantes es olvidar inicializar la lista en el constructor. Si no hacen `this.empleados = new ArrayList<>()`, la variable queda en null y obtendrán un `NullPointerException` al intentar usarla. Este es un error que verán mil veces, ¡aprendan a identificarlo!

El Patrón ABM: Alta, Baja y Modificación



No es suficiente tener la lista declarada; necesitamos métodos para interactuar con ella de forma segura y controlada. Aquí aplicamos uno de los principios fundamentales de la Programación Orientada a Objetos: el **Encapsulamiento**.

La lista es `private`, lo que significa que nadie desde fuera de la clase puede tocarla directamente. Todo acceso debe pasar por métodos públicos que nosotros controlamos. Esto nos da el poder de validar, registrar o denegar operaciones según las reglas de negocio.

Métodos de Gestión de la Lista

Agregar Empleado

```
public void
agregarEmpleado(Empleado e) {

    this.empleados.add(e);
}
```

Usa el método `.add()` del `ArrayList` para incorporar un nuevo elemento al final de la lista.

Eliminar Empleado

```
public void
quitarEmpleado(Empleado e) {

    this.empleados.remove(e);
}
```

Usa el método `.remove()` que busca el objeto y lo elimina de la lista si existe.

Contar Empleados

```
public int
getCantidad() {
    return
    this.empleados.size();
}
```

Usa `.size()` para obtener la cantidad actual de elementos en la lista.

 **Advertencia crítica sobre el Encapsulamiento:** Jamás hagan un `getEmpleados()` que devuelva la lista "cruda" (`return this.empleados`), porque romperían el encapsulamiento. Cualquiera podría borrar empleados desde afuera sin control alguno. Lo ideal es devolver una copia o un iterador. Aunque por ahora en la cursada permitiremos el "getter" simple, sean conscientes de este riesgo en proyectos profesionales.

Comparación: Arrays vs ArrayList

Arrays Tradicionales



- Tamaño fijo declarado al crearlos
- Acceso directo por índice [i]
- No pueden crecer ni reducirse
- Más eficientes en memoria
- Sintaxis: `Empleado[]`
`arr = new Empleado[10]`

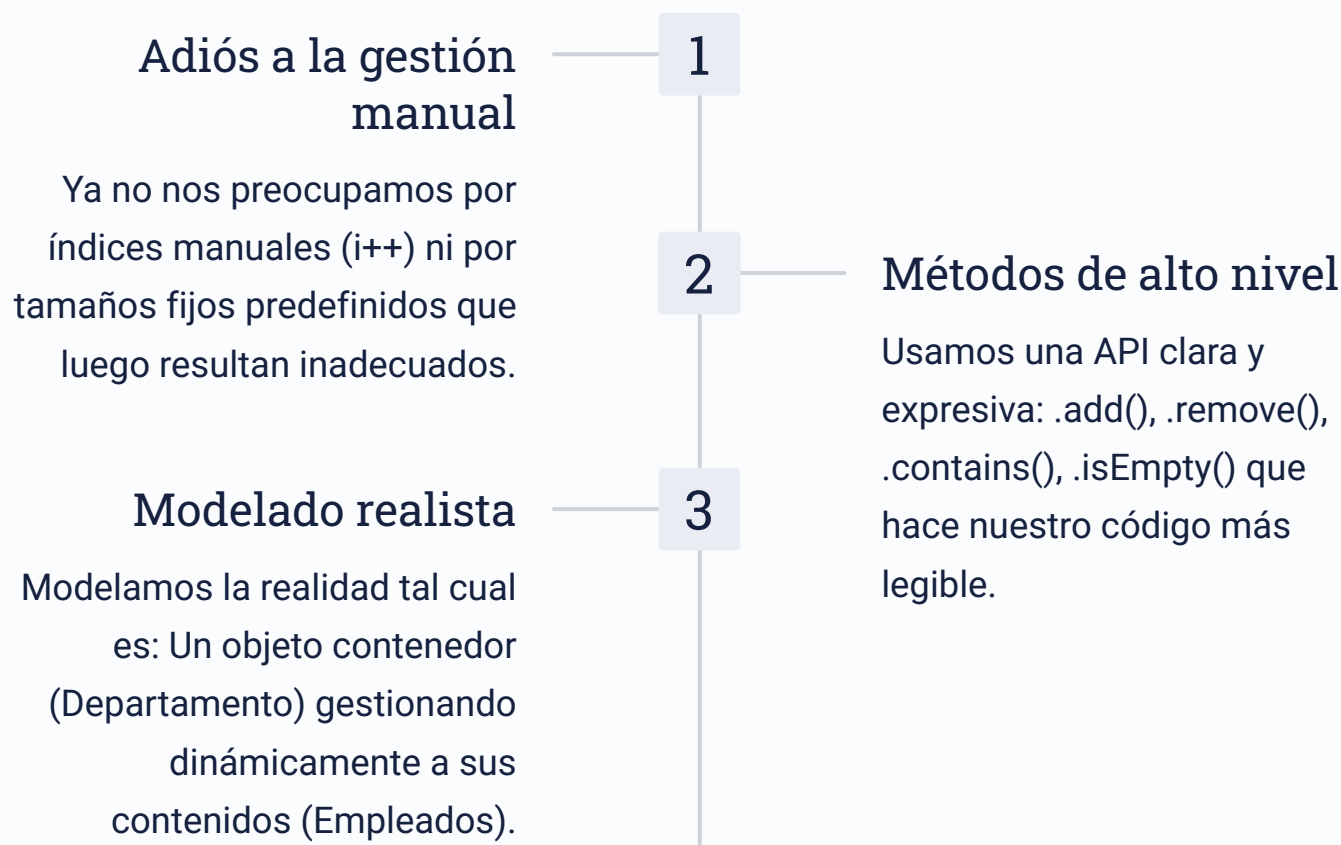
ArrayList Dinámico

- Tamaño flexible y dinámico
- Métodos de alto nivel: `.add()`, `.remove()`, `.get()`
- Crece automáticamente según su necesidad
- Pequeña sobrecarga de memoria debido a su flexibilidad
- Sintaxis: `ArrayList<Empleado>`
`lista = new ArrayList<>()`

La elección entre uno u otro depende del contexto. Si saben con certeza absoluta que necesitan exactamente 3 coordenadas (x, y, z) o 7 días de la semana, un array puede ser apropiado. Pero en la inmensa mayoría de casos reales, donde los datos son variables, **ArrayList es la opción correcta.**

La Nueva Forma de Programar

Con ArrayList, cambiamos fundamentalmente nuestra forma de pensar y programar. Es un salto cualitativo que marca la diferencia entre código frágil y código profesional.



Esta estructura de pensamiento es la base de lo que verán en Bases de Datos relacionales, en frameworks de desarrollo web, en arquitecturas empresariales. La relación 1:N aparece en todos lados: un Cliente tiene muchos Pedidos, un Pedido tiene muchos Items, una Universidad tiene muchos Estudiantes.

Dominar ArrayList y las relaciones 1:N es dominar el 80% de la programación orientada a objetos real. ¡Ahora a practicar con los ejercicios de la actividad!